

Transformação da Frequência de Variáveis

Guia rápido

O repositório [Variables_Frequency_Transforming.R](#) disponibiliza uma função auxiliar para transformação da frequência temporal de séries de dados. Seu principal objetivo é permitir a adequação de variáveis para diferentes frequências de análise, de acordo com o tratamento necessário para cada tipo de informação.

A função [cumulative_transform](#) permite realizar diferentes formas de transformação dos dados, incluindo **soma**, **média**, **seleção do período final**, **taxa acumulada** e **diferença de valores acumulados**.

As transformações podem ser realizadas para as frequências **mensal**, **bimestral**, **trimestral** e **semestral**. Além disso, a função permite preservar agrupamentos existentes na base e, opcionalmente, modificar a identificação da variável temporal para representar a nova frequência.

As próximas seções apresentam os argumentos utilizados pela função, os tipos de transformação disponíveis e exemplos de sua aplicação.

Cumulative_Transform

A função apresenta a seguinte estrutura:

```
cumulative_transform(  
  transform_type,  
  frequency,  
  dataset,  
  groupby_variables = NULL,  
  change_date_format = FALSE  
)
```

Para realizar uma transformação, o usuário precisa informar três argumentos principais: **transform_type**, **frequency** e **dataset**. Esses argumentos correspondem, respectivamente, ao tipo de transformação que será realizada, à frequência temporal desejada e à base de dados

que será transformada.

A função também possui dois argumentos opcionais. O argumento **groupby_variables** permite indicar uma ou mais variáveis que deverão ser preservadas como grupos durante a transformação. Já **change_date_format** determina se a coluna temporal será convertida para uma identificação correspondente à nova frequência.

Atenção!

- A base informada em **dataset** deve possuir uma coluna denominada **data**;
- A coluna **data** deve estar em formato compatível com datas no R;
- O valor padrão de **groupby_variables** é NULL;
- O valor padrão de **change_date** é FALSE.

Tipos de Transformação

O argumento **transform_type** define o tratamento aplicado às variáveis numéricas durante a transformação da frequência. A escolha deve considerar a natureza da variável analisada.

Os tipos de transformação disponíveis são:

- **soma** ("soma" ou "sum"): soma os valores pertencentes ao mesmo período;
- **media** ("media" ou "mean"): calcula a média dos valores pertencentes ao mesmo período;
- **periodo_final** ("periodo_final" ou "final_period"): mantém a observação correspondente ao período final de cada intervalo;
- **tx_acumulada** ("tx_acumulada" ou "cumulative_rate"): calcula a taxa acumulada a partir das taxas percentuais observadas dentro do período;
- **diff_acumulado** ("diff_acumulado" ou "cumulative_diff"): calcula a diferença entre valores acumulados de períodos consecutivos.

A transformação **periodo_final** é especialmente útil para variáveis de estoque, nas quais o valor de interesse corresponde à posição observada ao final de determinado período. Já **tx_acumulada** é indicada para taxas que precisam ser compostas ao longo do intervalo, enquanto **diff_acumulado** permite obter valores do período a partir de séries originalmente apresentadas de forma acumulada.

Frequências Disponíveis

O argumento **frequency** determina a frequência temporal desejada para os dados resultantes. Atualmente, a função permite trabalhar com quatro frequências.

- **Mensal**: "mensal" ou "monthly";
- **Bimestral**: "bimestral" ou "bimonthly";
- **Trimestral**: "trimestral" ou "quartely";
- **Semestral**: "semestral" ou "halfyear".

A frequência escolhida é utilizada em conjunto com **transform_type**, que define o tratamento aplicado aos dados durante a transformação.”

Alteração da Variável de Data

O argumento **change_date_format** controla a forma de apresentação da variável temporal após a transformação. Por padrão, seu valor é **FALSE**.

Quando **change_date_format = TRUE**, os valores da coluna **data** são convertidos para identificadores textuais correspondentes à frequência selecionada.

Exemplos de identificação:

- Mensal: 2026_M01, 2026_M02, ..., 2026_M12;
- Bimestral: 2026_B1, 2026_B2, ..., 2026_B6;
- Trimestral: 2026_Q1, 2026_Q2, ..., 2026_Q4;
- Semestral: 2026_H1 e 2026_H2.

Aplicação: Séries de Dados do Banco Central

A função **cumulative_transform** é utilizada neste exemplo para transformar a frequência de variáveis extraídas diretamente do Banco Central. As séries percorrem o intervalo mensal compreendido entre janeiro/2015 e dezembro/2025. Assuma que precisamos das séries na frequência trimestral. As variáveis em questão, bem como a regra de agregação implementada em cada uma, são:

- Exportação (Soma);
- Importação (Soma);
- Selic Mensal Acumulada Anualizada (Final de Período);
- Inflação IPCA (Taxa Acumulada).

```
# ===== #
# === Libraries === #
# ===== #
source('https://raw.githubusercontent.com/paulo-icaro/Bacen_API/main/bacen_query.R')

# ===== #
# === Data Extraction === #
# ===== #

# --- Previous Info --- #
cod_bacen_series = c('13093', '13094', '4189', '433')
name_bacen_series = c('exportacao', 'importacao', 'selic_mensal_acum_anualiz',
↳ 'inflação_ipca')
start_date = '01/01/2015'
end_date = '31/12/2025'
#end_date = format(Sys.Date(), '%d/%m/%Y')

# --- Extraction --- #
```

```

bacen_dataset = bacen_query(cod_bacen_series, name_bacen_series, start_date,
  ↪ end_date)

# --- Date Adjustment --- #
bacen_dataset = bacen_dataset %>% mutate(data = as.Date(data, tryFormats =
  ↪ c('%d/%m/%Y')))
bacen_dataset[c(-1)] = lapply(X = bacen_dataset[c(-1)], FUN = as.numeric)
head(bacen_dataset, n = 10)

```

	data	exportacao	importacao	selic_mensal_acum_anualiz	inflação_ipca
1	2015-01-01	99525	647477	11.82	1.24
2	2015-02-01	74722	148715	12.15	1.22
3	2015-03-01	78281	215278	12.58	1.32
4	2015-04-01	74072	211604	12.68	0.71
5	2015-05-01	72148	252533	13.15	0.74
6	2015-06-01	80672	157724	13.58	0.79
7	2015-07-01	88933	223834	13.69	0.62
8	2015-08-01	80011	136616	14.15	0.22
9	2015-09-01	95899	243164	14.15	0.54
10	2015-10-01	98567	146785	14.15	0.82

```

# ===== #
# === Libraries === #
# ===== #
source('https://raw.githubusercontent.com/paulo-icaro/Variables_Frequency_Transforming/main/variables

# ===== #
# === Transforming Data Frequency === #
# ===== #
bacen_dataset_bimonthly_sum = cumulative_transform('soma', 'bimestral',
  ↪ bacen_dataset[c(1:3)])
bacen_dataset_bimonthly_end = cumulative_transform('periodo_final', 'bimestral',
  ↪ bacen_dataset[c(1, 4)])
bacen_dataset_bimonthly_cum = cumulative_transform('tx_acumulada', 'bimestral',
  ↪ bacen_dataset[c(1, 5)])
bacen_dataset_bimonthly = left_join(x = bacen_dataset_bimonthly_sum, y =
  ↪ bacen_dataset_bimonthly_end, by = 'data')
bacen_dataset_bimonthly = left_join(x = bacen_dataset_bimonthly, y =
  ↪ bacen_dataset_bimonthly_cum, by = 'data')

print(bacen_dataset_bimonthly)

```

```

# A tibble: 66 x 5
  data      exportacao importacao selic_mensal_acum_anualiz inflação_ipca

```

	<date>	<dbl>	<dbl>	<dbl>	<dbl>
1	2015-02-01	174247	796192	12.2	2.48
2	2015-04-01	152353	426882	12.7	2.04
3	2015-06-01	152820	410257	13.6	1.54
4	2015-08-01	168944	360450	14.2	0.841
5	2015-10-01	194466	389949	14.2	1.36
6	2015-12-01	201410	289646	14.2	1.98
7	2016-02-01	158552	288361	14.2	2.18
8	2016-04-01	155060	308469	14.2	1.04
9	2016-06-01	159064	1577978	14.2	1.13
10	2016-08-01	212525	661225	14.2	0.962

i 56 more rows

Outros Exemplos de Aplicação

Média

Para variáveis cuja agregação deve representar o valor médio observado no período:

```
dados_bimestrais = cumulative_transform(
    "media",
    "bimestral",
    dados,
    change_date_format = FALSE/TRUE
)
```

Diferença Acumulada

Para variáveis apresentadas de forma acumulada, quando se deseja obter a diferença correspondente ao período:

```
dados_bimestrais = cumulative_transform(
    "diff_acumulado",
    "bimestral",
    dados,
    change_date_format = FALSE/TRUE
)
```

Dependências

O funcionamento de `cumulative_transform` depende dos pacotes `dplyr` e `lubridate`. O próprio script verifica o carregamento dessas bibliotecas antes da execução da função.

Além disso, para o correto uso da função, o arquivo deve ser previamente carregado no ambiente de trabalho localmente ou via Github.

```
source("Variables_Frequency_Transforming.R")
```